

Informe de Seguridad en APIs RESTful: Análisis y Mitigación de Vulnerabilidades OWASP

Seguridad y Mitigación de Riesgos en API PHP

Objetivo del ejercicio

Analizar un fragmento vulnerable de código PHP correspondiente a una API RESTful, identificar vulnerabilidades asociadas al estándar **OWASP Top 10**, y aplicar medidas de corrección mediante técnicas seguras de desarrollo web. El foco está en mitigar riesgos como:

- Broken Access Control
- Inyección SQL
- Cross-Site Scripting (XSS)
- Autenticación y autorización insegura

Escenario Proporcionado

Se entregó el siguiente fragmento vulnerable de código PHP que permite eliminar usuarios por medio de un parámetro GET:

```
<?php
$id = $_GET['id'];
if ($_GET['role'] === 'admin') {
    $conexion->query("DELETE FROM usuarios WHERE id = $id");
    echo "Usuario eliminado";
} else {
    echo "Acceso denegado";
}
?>
```

Análisis de Vulnerabilidades

a) Vulnerabilidades detectadas en el código original:

Vulnerabilidad	Presente
Inyección SQL	✓
Broken Access Control	✓
Cifrado débil	✗
Falta de autenticación/autorización	✓

b) Riesgos potenciales para un atacante:

- ✓ Eliminar usuarios sin autenticación
- ✓ Manipular el ID en la URL para eliminar cualquier usuario
- ✗ Cambiar el fondo del sitio (irrelevante para este contexto)

Reescritura Segura del Código

Se propone una versión corregida del código original con las siguientes medidas de mitigación:

- Verificación de autenticación mediante sesión activa (`$_SESSION`)
- Validación de rol con privilegios desde servidor
- Uso de **consultas preparadas (prepared statements)** para evitar inyecciones

- Sanitización del parámetro `id`

Código Seguro

```
<?php

session_start();

// Verificación de sesión activa y rol de administrador

if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {

    http_response_code(403);

    echo json_encode(["error" => "Acceso denegado"]);

    exit;

}

// Validación de entrada

if (!isset($_GET['id']) || !is_numeric($_GET['id'])) {

    http_response_code(400);

    echo json_encode(["error" => "Parámetro inválido"]);

    exit;

}

$id = intval($_GET['id']); // Sanitizar input

// Conexión a base de datos

$conexion = new mysqli("localhost", "usuario", "contraseña", "basedatos");

if ($conexion->connect_error) {
```

```
http_response_code(500);

echo json_encode(["error" => "Error de conexión"]);

exit;

}

// Consulta preparada segura

$stmt = $conexion->prepare("DELETE FROM usuarios WHERE id = ?");

$stmt->bind_param("i", $id);

if ($stmt->execute()) {

    echo json_encode(["mensaje" => "Usuario eliminado"]);

} else {

    http_response_code(500);

    echo json_encode(["error" => "Error al eliminar"]);

}

$stmt->close();

$conexion->close();

?>
```

Simulación con Postman / cURL

Se recomienda realizar pruebas comparativas del endpoint original y el seguro bajo los siguientes escenarios:

Escenario	Resultado Inseguro	Resultado Seguro
Sin sesión activa	Usuario eliminado	403 Acceso denegado
Rol no autorizado	Usuario eliminado	403 Acceso denegado
ID manipulado por el cliente	Eliminación arbitraria	Eliminación solo si el ID existe

Reflexión Final

¿Por qué usar roles desde sesión en vez de parámetros GET?

Porque las sesiones del lado servidor son confiables y no manipulables por el cliente. Usar `$_GET['role']` permite que cualquier usuario se declare "admin" manualmente, comprometiendo la seguridad.

¿Por qué es peligroso construir consultas directamente con valores de entrada?

Permite ataques de **inyección SQL**, donde el atacante puede ejecutar comandos arbitrarios y potencialmente borrar, robar o modificar datos sensibles.

Conclusión

Este ejercicio ha demostrado cómo un pequeño fragmento de código puede representar múltiples riesgos cuando no se aplican buenas prácticas de desarrollo seguro. Mediante el uso de **verificación de autenticación robusta**, **control de acceso estricto** y **consultas preparadas**, se logró mitigar correctamente las vulnerabilidades más críticas del ejemplo.